

Arrays, Hashing, and Two-Pointer Reasoning

Arrays give indexed access and locality; hashing trades memory for keyed lookup; two pointers exploit ordering or a maintained partition. The useful skill is recognizing which information must be retained and which candidate region can be discarded safely.

For: Learners who know loops and collections and want to understand common linear and near-linear solutions without memorizing a separate trick for every array problem.

Suggested pace: 2 weeks of focused practice | Foundational DSA

What you will be able to do

- Choose between sorting, hashing, and pointer movement using the input contract.
- Handle duplicates, multiplicities, and index-versus-value requirements correctly.
- State the invariant behind complement lookup and sorted two-pointer search.
- Analyze expected hash-table cost and auxiliary space honestly.

Build the right mental model

Values and positions are different

A problem may ask for original indexes, unique values, or all valid pairs. Sorting can destroy original positional information unless it is retained. Duplicates may be legitimate separate positions rather than redundant values to discard.

Hashing stores the missing information

For complement search, retain previously seen values and their indexes. For frequency questions, store counts rather than membership. Expected constant-time operations depend on the implementation and assumptions; do not describe every hash lookup as a worst-case guarantee.

Pointer movement needs a proof

In a sorted sum problem, if the sum is too small, reducing the right value cannot help, so move the left pointer. Without ordering, this elimination argument is invalid. An in-place compaction pointer instead represents the boundary of already processed valid output.

Study with evidence.

For each exercise, write the required output and duplicate policy first. Solve a small baseline, then explain what the map or pointer invariant eliminates. Test repeated values and an answer involving the same numeric value at two different positions.

Days 1-3: Array contracts

Practice traversal, stable compaction, prefix/suffix accumulation, and in-place updates. Distinguish logical length from capacity and check whether input mutation is allowed. Trace read and write positions on paper.

Deliverable and check: A compaction example that preserves order and never overwrites unread data.

Days 4-6: Maps and counts

Study complement lookup, duplicates, anagrams, and frequency grouping. Decide whether the map stores a count, earliest index, latest index, or a list of positions. Explain memory growth and missing-key behavior.

Deliverable and check: Several examples where using a set instead of a count or index map would be wrong.

Days 7-10: Sorted pointer methods

Practice pair sums, deduplicated triples, and merging sorted sequences. Justify each pointer movement and where duplicates should be skipped. Preserve original indexes if the output requires them.

Deliverable and check: A sorted-pair proof and a merge trace that handles equal values and empty inputs.

Days 11-14: Mixed variants

Change one requirement at a time: return counts, preserve order, forbid mutation, or allow repeated results. Compare sorting and hashing under those constraints and run an unseen timed problem.

Deliverable and check: A decision table connecting contract changes to data-structure and complexity changes.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Find a pair without reusing the same element

Given values [3, 3, 4] and target 6, return two distinct indexes. A value-only set can show that 3 exists, but careless ordering can pair an element with itself.

A worked approach

1. Scan from left to right and look up target minus current value before inserting the current index. At index zero, the complement is absent, so record value 3 at index zero.
2. At index one, the complement 3 is already recorded at index zero. Return indexes zero and one. The invariant is that the map contains only earlier positions, so the two indexes are necessarily distinct.
3. Compare with sorting indexed pairs and using two pointers. That method costs sorting time but can be useful for other output contracts. Explain why simply sorting raw values would lose original indexes.

Expected result

The complement method uses expected $O(n)$ time and $O(n)$ auxiliary space, with a clear distinct-index guarantee. It does not require that values themselves be distinct.

Verify the result

- Does target 6 with input [3] correctly report no pair?
- Are duplicate values handled according to the requested output?
- Does the method preserve original indexes when required?

Extend the exercise

Return the number of all index pairs instead of the first pair. Replace the stored index with a frequency count and explain why each earlier matching occurrence contributes one new pair.

Make the learning stick.

When is sorting better than hashing?

Sorting may support ordered output, range reasoning, duplicate grouping, or lower auxiliary-memory requirements depending on the language. Compare the full contract, including whether mutation and loss of original order are acceptable.

Why does a set fail for some anagram or pair-count questions?

Membership does not preserve multiplicity. Two strings can use the same character set with different counts, and repeated values can create several distinct index pairs. Store the information the output actually depends on.

Can two pointers always make an array problem linear?

No. The method needs a monotone elimination or partition invariant. Without that structure, moving a pointer may discard a valid solution. Name the proof, not just the technique.

Avoid these traps

- Inserting before complement lookup and accidentally reusing an index.
- Discarding duplicates that are meaningful to the output.
- Claiming worst-case $O(1)$ hash operations without qualification.

Ready for the next step?

- The map value's meaning is explicit.
- Every pointer movement has a correctness argument.
- Mutation and original-order requirements are respected.
- Duplicate and minimal-input cases have been traced.

Reliable next reads

- [MIT algorithms course](#)
- [Python collections documentation](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.